UNIVERSIDADE ESTADUAL PAULISTA
"JÚLIO DE MESQUITA FILHO"

Campus de Sorocaba
ICTS Instituto de Ciências e Tecnologia

Trabalho 2

Redes de Comunicação Industrial

Discente: Luiz Henrique Corrêa Monteiro, Paulo de Melo Pereira R. Ramalho
Orientador: Prof. Dr. Eduardo Godoy Paciência

Sorocaba - SP
2026

Luiz Henrique Corrêa Monteiro, Paulo de Melo Pereira R. Ramalho

Programação Remota, Comunicação e Visualização Web usando API Rest HTTP

Trabalho apresentado ao Instituto de Ciências e Tecnologia da Unesp, como parte dos requisitos para a conclusão da disciplina em Redes de Comunicação Industrial.

Orientador: Prof. Dr. Eduardo Godoy Paciência

E-mail dos alunos autores:
lh.monteiro@unesp.br
paulo.ramalho@unesp.br

Sorocaba - SP
2026

Índice

1 Introdução **2**

2 Dificuldades e impressões **2**

3 Materiais e métodos **3**
3.1 Descrição do módulo 3
3.2 Descrição da arquitetura de rede 3
3.3 Metodologia de implementação 4

4 Programação **5**
4.1 Programação do ESP32 5
4.2 Construção da interface Node-Red 7
4.3 Realização do OTA 9
4.4 Desenvolvimento extra 11

5 Conclusão **11**

1 Introdução

Hoje em dia os sistemas modernos estão cada vez mais integrados entre si, e para fazer essa troca de informações uma das formas mais viáveis é o uso de **HTTP API Rest**. A ideia central do projeto é utilizar uma placa de desenvolvimento *ESP32* com a tecnologia LoRa para ler dados de um sensor que mede umidade, altitude e temperatura, além de fazer a leitura do estado de um botão e controlar o acionamento de um LED interno.

Essas informações coletadas são enviadas pela internet via API REST, para um dashboard no Node-Red. Além disso, esse modo de comunicação permite que o Node-Red também envie comandos para a placa ESP32. Além do Node-Red, foi desenvolvido como extra uma interface HTML5, que permite outras formas de visualização, bem como um simulador digital da placa ESP32, por fim, os dados são salvos em um banco de dados SQL.

A utilização dessa estrutura é bastante ampla, podendo ser empregada tanto no ramo industrial, bem como para automação residencial, profissional, etc.

2 Dificuldades e impressões

Durante todo o processo de construção do projeto, algumas dificuldades foram enfrentadas, dentre elas a seleção da fonte de dados (ESP32 Real ou simulador). A questão é que mudar a fonte de dados no frontend não implica, necessariamente, alterar a fonte utilizada pelo backend. Para solucionar esse problema, foi necessária a implementação de uma lógica de seleção da fonte ativa, garantindo que os dados apresentados e armazenados correspondessem ao dispositivo selecionado pelo usuário.

Além disso, outro ponto que exigiu atenção foi a integração do Node-RED com a API. Como o Node-RED, o frontend, o simulador e o ESP32 realizam requisições ao backend, foi necessário garantir que todos utilizassem o mesmo endereço da API e a mesma identificação do dispositivo ativo. Durante os testes, diferenças entre o endereço utilizado localmente (`127.0.0.1`) e o endereço utilizado pelo ESP32 na rede fizeram com que diferentes componentes não apresentassem o mesmo estado da aplicação. A padronização do acesso ao backend foi, portanto, fundamental para manter a comunicação consistente entre os diferentes módulos.

A implementação da comunicação com o ESP32 real também apresentou dificuldades adicionais em relação ao simulador. Enquanto o simulador executado no computador possui acesso direto aos serviços locais, o ESP32 depende da comunicação pela rede Wi-Fi. Dessa forma, foi necessário verificar aspectos como o endereço IP do servidor, a disponibilidade da porta utilizada pela API e a conectividade entre o microcontrolador e o computador. Os códigos de resposta HTTP fornecidos pela API foram particularmente úteis durante essa etapa, permitindo identificar situações em que a comunicação estava ocorrendo, mas a requisição era rejeitada por incompatibilidade entre o dispositivo e a fonte de dados selecionada.

Outro aspecto importante foi a implementação da atualização remota do firmware por meio de OTA (*Over-the-Air*). Essa funcionalidade exigiu que o ESP32 estivesse corretamente conectado à rede e pudesse ser identificado pelo computador antes do envio de uma nova versão do firmware. Apesar das dificuldades iniciais de configuração e detecção do dispositivo pela rede, o recurso mostrou-se especialmente útil por permitir futuras alterações no firmware sem a necessidade de conexão física direta ao microcontrolador.

De maneira geral, as dificuldades encontradas durante o desenvolvimento contribuíram para uma melhor compreensão da integração entre sistemas embarcados, aplicações web e serviços de backend. Os testes realizados ao longo do desenvolvimento foram, portanto, essenciais para identificar problemas de comunicação e tornar a aplicação mais integrada e confiável.

3 Materiais e métodos

3.1 Descrição do módulo

A placa utilizada para a prototipagem e desenvolvimento do projeto, foi um módulo que integra um microcontrolador **ESP32**, equipado com um display LCD, e uma antena de LoRa (*Long Range*). Além disso, o kit ainda contempla um sensor **Bosch BME 280**, o qual apresenta comunicação serial com o ESP32, via I2C (*Inter-Integrated Circuit*) ou SPI (*Serial Peripheral Interface*), e é capaz de analisar as seguintes variáveis:

- pressão;
- temperatura;
- umidade;
- altitude.

O ESP32 utilizado, apresenta a seguinte configuração de *pinout*:

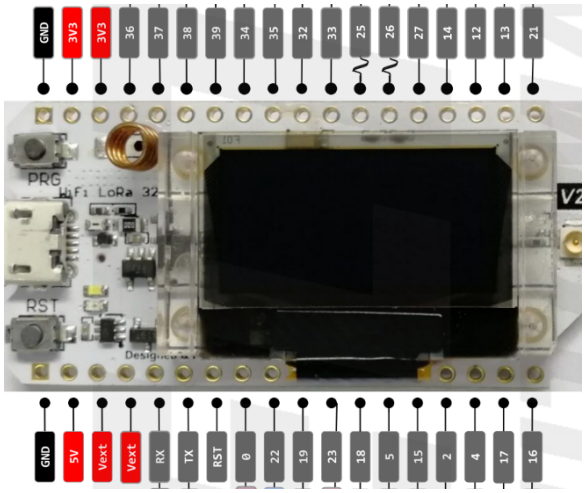


Figura 1: Fotografia do *pinout* da placa ESP32 LoRa.

Fonte: (**heltec_wifi_lora_32_v2**), 2026.

3.2 Descrição da arquitetura de rede

A arquitetura utilizada no projeto foi desenvolvida buscando separar as diferentes partes do sistema, de modo que cada uma seja responsável por uma determinada função. De maneira geral, a estrutura é formada pelo ESP32, uma API REST desenvolvida em Python, um banco de dados PostgreSQL, a interface Node-Red e, como desenvolvimento extra, uma interface Web e um simulador do ESP32.

A comunicação principal ocorre através do protocolo HTTP. O ESP32 se conecta à rede Wi-Fi e realiza periodicamente o envio das informações coletadas pelo BME280 para o *backend*. Este servidor foi desenvolvido utilizando o *framework* FastAPI e é executado na porta 8000 do computador responsável pelo sistema.

Os dados enviados pelo ESP32 são compostos pela identificação do dispositivo, temperatura, umidade, pressão e estado do botão físico. No firmware também é calculada a altitude

através da pressão atmosférica, entretanto, para o armazenamento principal do sistema foram utilizados os valores de temperatura, umidade e pressão.

O fluxo simplificado de comunicação pode ser representado da seguinte maneira:

$$\text{ESP32} \rightarrow \text{API REST} \rightarrow \text{PostgreSQL}$$

Além do envio de dados, a comunicação ocorre também no sentido contrário para o controle do LED. Neste caso, o Node-Red ou a interface Web envia uma requisição para a API solicitando a alteração do estado do LED. O ESP32 consulta periodicamente este estado através da API e atualiza sua saída digital de acordo com o valor recebido.

Dessa maneira, não é necessário que o Node-Red se comunique diretamente com o ESP32 para todas as operações. A API funciona como um intermediário entre os diferentes elementos do sistema, o que facilita a troca da fonte de dados e também a utilização do simulador.

Para diferenciar o equipamento físico do simulado foram utilizados dois identificadores. O ESP32 real utiliza o *device id* `esp32-001`, enquanto o simulador utiliza `esp32-simulator-001`. A fonte que está sendo utilizada pode ser selecionada pela própria interface, evitando que os dois dispositivos enviem informações simultaneamente para o sistema.

3.3 Metodologia de implementação

A implementação foi realizada de forma modular, inicialmente desenvolvendo e testando as diferentes partes do sistema de maneira separada. Essa escolha foi feita principalmente para permitir que grande parte do projeto pudesse ser desenvolvido antes da utilização do ESP32 físico.

Primeiramente foi desenvolvido um simulador em Python, utilizando também o FastAPI. Esse programa reproduz as principais funcionalidades esperadas para o ESP32, gerando valores de temperatura, umidade e pressão, além de permitir a simulação do botão e do LED. Os valores apresentam pequenas variações durante a execução, buscando representar de maneira simples o comportamento de um sensor real.

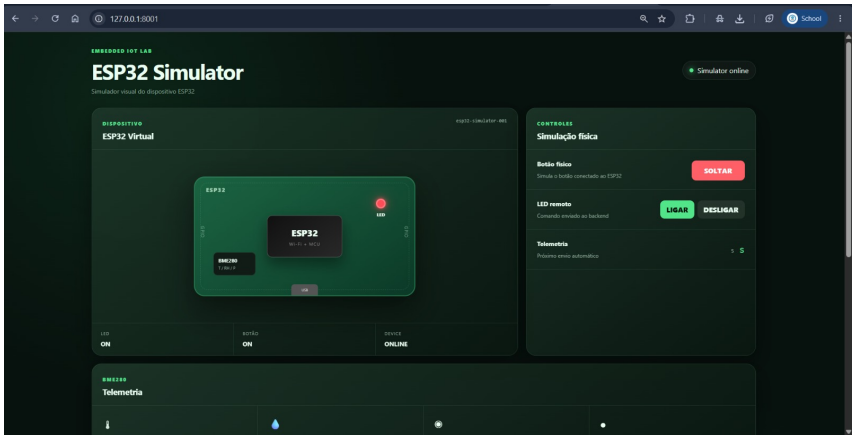


Figura 2: Captura de tela do simulador desenvolvido.

Após isso, foi desenvolvido o *backend* principal da aplicação. A API foi responsável por receber os dados enviados pelo simulador ou pelo ESP32, armazenar essas informações e disponibilizar rotas para consulta. Também foram implementadas rotas específicas para o controle do LED e para escolha entre a fonte simulada e o ESP32 real.

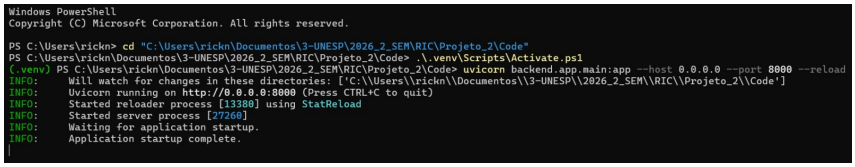
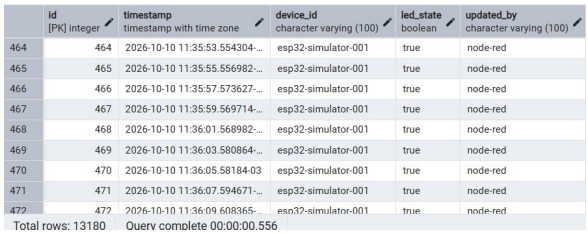


Figura 3: Captura de tela do backend rodando.

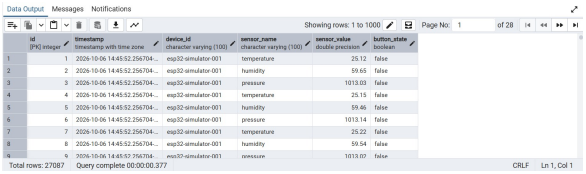
Para o armazenamento das informações foi utilizado o PostgreSQL. Foram criadas tabelas separadas para os dados dos sensores e para os estados do atuador. Cada medição armazenada possui um instante de tempo, identificação do dispositivo, nome do sensor, valor medido e estado do botão. Para o LED são armazenados seu estado, dispositivo relacionado, horário da alteração e a origem do comando.



	id [PK] integer	timestamp timestamp with time zone	device_id character varying (100)	led_state boolean	updated_by character varying (100)
464	464	2026-10-10 11:35:53.554304-...	esp32-simulator-001	true	node-red
465	465	2026-10-10 11:35:55.556982-...	esp32-simulator-001	true	node-red
466	466	2026-10-10 11:35:57.573627-...	esp32-simulator-001	true	node-red
467	467	2026-10-10 11:35:59.569714-...	esp32-simulator-001	true	node-red
468	468	2026-10-10 11:36:01.568982-...	esp32-simulator-001	true	node-red
469	469	2026-10-10 11:36:03.580864-...	esp32-simulator-001	true	node-red
470	470	2026-10-10 11:36:05.58184-03	esp32-simulator-001	true	node-red
471	471	2026-10-10 11:36:07.594671-...	esp32-simulator-001	true	node-red
472	472	2026-10-10 11:36:09.608365-...	esp32-simulator-001	true	node-red

Total rows: 13180 Query complete 00:00:00.556

(a) Primeira tabela.



	id [PK] integer	timestamp timestamp with time zone	device_id character varying (100)	sensor_name character varying (100)	sensor_value double precision	button_state boolean
1	1	2026-10-06 14:45:52.256704-...	esp32-simulator-001	temperature	25.12	false
2	2	2026-10-06 14:45:52.256704-...	esp32-simulator-001	humidity	59.65	false
3	3	2026-10-06 14:45:52.256704-...	esp32-simulator-001	pressure	1013.03	false
4	4	2026-10-06 14:45:52.256704-...	esp32-simulator-001	temperature	25.15	false
5	5	2026-10-06 14:45:52.256704-...	esp32-simulator-001	humidity	59.46	false
6	6	2026-10-06 14:45:52.256704-...	esp32-simulator-001	pressure	1013.14	false
7	7	2026-10-06 14:45:52.256704-...	esp32-simulator-001	temperature	25.22	false
8	8	2026-10-06 14:45:52.256704-...	esp32-simulator-001	humidity	59.54	false
9	9	2026-10-06 14:45:57.256704-...	esp32-simulator-001	pressure	1013.07	false

Total rows: 27087 Query complete 00:00:00.377

(b) Segunda tabela.

Figura 4: Capturas de tela das tabelas de dados SQL.

Com o funcionamento do simulador, da API e do banco de dados, foi possível desenvolver a interface de monitoramento sem depender inicialmente do hardware. Posteriormente o firmware do ESP32 foi programado utilizando o mesmo padrão de comunicação REST empregado pelo simulador. Desta forma, a troca entre o equipamento simulado e o equipamento físico não exige alterações nas rotas principais utilizadas pelo restante do sistema.

Por fim foi realizada a integração com o Node-Red, reunindo a visualização das variáveis, histórico dos sensores, estado do botão, controle do LED e seleção da fonte de dados em uma única interface.

4 Programação

Todo o código desenvolvido e utilizado, está presente no repositório do Git-Hub, e pode ser acessado clicando AQUI.

O projeto foi separado em diferentes diretórios, sendo os principais destinados ao firmware do ESP32, simulador, *backend*, banco de dados, interface Web e Node-Red. Essa divisão foi utilizada para facilitar principalmente os testes e alterações durante o desenvolvimento.

4.1 Programação do ESP32

A programação do ESP32 foi desenvolvida em C++, utilizando as bibliotecas disponibilizadas para o ambiente Arduino. O código realiza a leitura do sensor BME280, leitura do botão físico, controle do LED, atualização do display OLED, conexão Wi-Fi, comunicação HTTP com o servidor e atualização remota por OTA.

O Código 1 apresenta as principais configurações utilizadas no firmware do ESP32, incluindo as bibliotecas empregadas, a configuração da rede Wi-Fi, o endereço do servidor backend, a identificação do dispositivo, os parâmetros de atualização OTA e os pinos utilizados pelos periféricos.

```
/*
============================================================
PROJETO 2 - RIC
ESP32 + BME280 + OLED + REST + Wi-Fi + OTA
============================================================
*/

#include <Arduino.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <WebServer.h>
#include <ArduinoOTA.h>
#include <ESPmDNS.h>
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>
#include <SSD1306.h>

// ============================================================
// CONFIGURACAO
// ============================================================

const char* WIFI_SSID = "labautomacao";
const char* WIFI_PASSWORD = "********";

// IP do notebook na mesma rede do ESP32
const char* BACKEND_HOST = "200.145.26.14";
const int BACKEND_PORT = 8000;

// Identificacao do ESP32
const char* DEVICE_ID = "esp32-001";
const char* HOSTNAME = "esp32-001";

// Senha para atualizacoes OTA
const char* OTA_PASSWORD = "********";

// ============================================================
// HARDWARE
// ============================================================

#define LEDPIN 25
#define BUTTON 0

// BME280
#define I2C_SDA 21
#define I2C_SCL 22
#define SEALEVELPRESSURE_HPA 1029.9

// OLED
#define OLED_I2C_ADDR 0x3C
#define OLED_RESET 16
#define OLED_SDA 4
#define OLED_SCL 15
```

Listing 1: Configuração inicial do firmware do ESP32.

Para comunicação com o BME280 foi utilizado o protocolo I2C. No programa foram definidos os pinos GPIO 21 para SDA e GPIO 22 para SCL. Durante a inicialização o código

procura o sensor inicialmente no endereço `0x76` e, caso ele não seja encontrado, é feita uma segunda tentativa utilizando o endereço `0x77`. Esse procedimento permite utilizar diferentes configurações encontradas nos módulos BME280.

As leituras de temperatura, umidade e pressão são feitas utilizando a biblioteca `Adafruit_BME280`. A altitude também é calculada localmente utilizando a pressão de referência ao nível do mar. O botão utilizado no projeto foi configurado no GPIO 0 como `INPUT_PULLUP`, enquanto o LED foi configurado no GPIO 25 como saída digital.

Além do sensor, foi utilizado o display OLED presente no módulo. O display é atualizado aproximadamente a cada segundo, apresentando temperatura, umidade, pressão, endereço IP do ESP32, estado do LED e do botão. Dessa maneira foi possível verificar algumas das principais informações diretamente no equipamento, sem depender do dashboard.



Figura 5: Fotografia do ESP32 juntamente com o dashboard Node-Red ao fundo, e a tela

Após a inicialização, o ESP32 tenta realizar a conexão com a rede Wi-Fi configurada. Caso a conexão seja perdida durante a execução, o próprio programa realiza novas tentativas de conexão, sem a necessidade de reiniciar a placa.

A telemetria é enviada para a API REST a cada 5 segundos através de uma requisição HTTP `POST`. Os dados são enviados utilizando o formato JSON, contendo o identificador do ESP32 e os valores medidos. No servidor, esses dados são validados e posteriormente armazenados no PostgreSQL.

Para o acionamento do LED foi utilizado um funcionamento um pouco diferente. O ESP32 realiza uma requisição `GET` aproximadamente a cada 2 segundos para consultar qual deve ser o estado do LED. Quando existe uma alteração feita pelo Node-Red ou pela interface Web, a nova informação é recebida pelo ESP32 e aplicada ao pino de saída.

Também foi implementado no próprio ESP32 um pequeno servidor HTTP local. Por meio dele é possível consultar o estado do equipamento, telemetria e também realizar testes de acionamento do LED diretamente pelo endereço IP da placa.

4.2 Construção da interface Node-Red

A interface principal de visualização e controle foi desenvolvida utilizando o Node-Red. A escolha dessa ferramenta permitiu organizar a comunicação através de blocos, além de criar um dashboard para acompanhar os dados sem a necessidade de acessar diretamente a API.

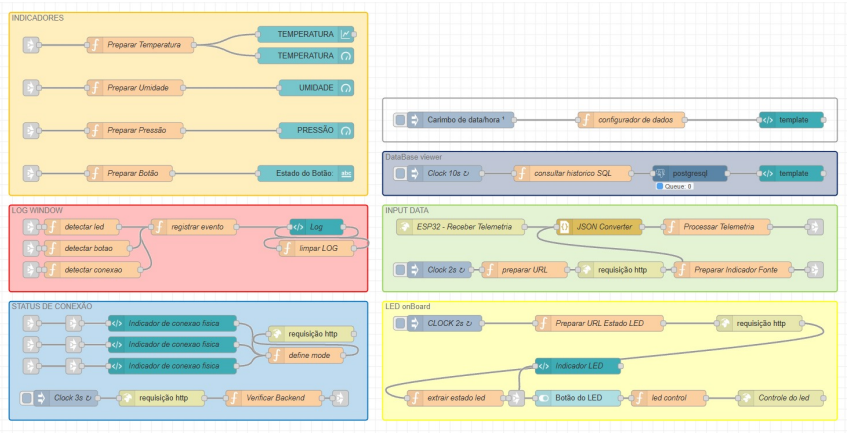
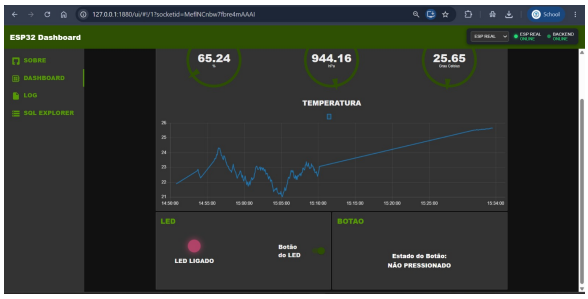


Figura 6: Captura de tela do Flow do node-red.

O fluxo desenvolvido consulta periodicamente a API REST e utiliza os dados retornados para atualizar os elementos presentes no dashboard. Foram criados indicadores para temperatura, umidade, pressão, estado do botão e estado do LED. Também foram adicionados gráficos para permitir a visualização dos valores registrados ao longo do tempo.



(a) Primeiro print do Dashboard principal.



(b) Segundo print do Dashboard principal.

Figura 7: Captura de tela dos Dashboards principais.

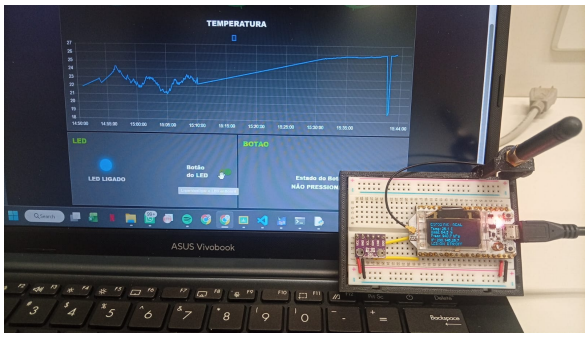
Para evitar que fosse necessário alterar manualmente os endereços das requisições durante os testes, foi implementado um seletor de fonte. Através desse seletor é possível escolher entre `SIMULATOR` e `ESP REAL`. Quando a seleção é alterada, o Node-Red modifica o identificador utilizado pelas demais requisições e informa a nova fonte para o *backend* através da rota `/api/source`.

Quando o simulador é escolhido, o identificador utilizado passa a ser `esp32-simulator-001`. Para o equipamento físico é utilizado `esp32-001`. O mesmo estado é compartilhado pelas diferentes partes do fluxo, permitindo que gráficos, indicadores e comandos utilizem sempre o dispositivo selecionado.

O controle do LED também foi integrado ao dashboard. Ao alterar o estado através da interface, é realizada uma requisição HTTP `POST` para a rota responsável pelo atuador. O novo estado fica registrado no servidor e posteriormente é consultado pelo ESP32. Também é realizada uma consulta periódica do estado atual, fazendo com que a interface permaneça sincronizada mesmo quando a alteração ocorre por outro ponto do sistema.



(a) Fotografia do ESP32 com o led desligado, sendo exibido o Dashboard no estado DESLIGADO.



(b) Fotografia do ESP32 com o led ligado, sendo exibido o Dashboard no estado LIGADO.

Figura 8: Fotografias do ESP32 com o led ligado e desligado.

Além dos dados atuais, o Node-Red possui comunicação com o banco PostgreSQL, permitindo consultar informações armazenadas anteriormente. Foram utilizados os módulos `node-red-dashboard` e `node-red-contrib-postgresql` para construção da interface e comunicação com o banco de dados.

Também foram implementados indicadores de conexão para informar se o *backend* e a fonte selecionada estão online ou offline. Isso foi importante durante os testes, principalmente para diferenciar um problema na comunicação do servidor de uma possível desconexão do ESP32.

4.3 Realização do OTA

Além da programação convencional pela porta USB, foi implementada a possibilidade de atualização do código através de OTA (*Over-The-Air*). Esse recurso permite enviar uma nova versão do programa para o ESP32 utilizando a própria rede Wi-Fi, sem que seja necessário manter a placa conectada fisicamente ao computador responsável pela programação.

Para isso foi utilizada a biblioteca `ArduinoOTA`. Durante a inicialização do ESP32 é definido um *hostname* para o dispositivo e uma senha de autenticação. Após a conexão com a rede Wi-Fi, o serviço OTA é inicializado e o ESP32 passa a aguardar possíveis atualizações.

```
1    // Configuracao OTA
2    void setupOTA() {
3        if (otaStarted || WiFi.status() != WL_CONNECTED) return;
4
5        ArduinoOTA.setHostname(HOSTNAME);
6        ArduinoOTA.setPassword(OTA_PASSWORD);
7
8        ArduinoOTA.onStart([]() {
9            Serial.println("OTA iniciado...");
10       });
11
12       ArduinoOTA.onEnd([]() {
13           Serial.println("\nOTA concluido.");
14       });
15
16       ArduinoOTA.onProgress([](unsigned int progress, unsigned int total)
     {
17           Serial.printf("OTA: %u%%\r", (progress * 100) / total);
18       });
19
20       ArduinoOTA.onError([](ota_error_t error) {
```

```
            Serial.printf("Erro OTA [%u]\n", error);
        });

        ArduinoOTA.begin();
        otaStarted = true;

        Serial.print("OTA pronto: ");
        Serial.print(HOSTNAME);
        Serial.println(".local");
    }
```

Listing 2: Configuração do serviço de atualização remota OTA no ESP32.

O dispositivo foi configurado com o nome `esp32-001`, podendo ser identificado na rede também através do serviço mDNS. Durante a execução normal, a função responsável pelo OTA é chamada continuamente no *loop* principal, juntamente com as funções de leitura dos sensores, servidor HTTP e envio dos dados.

O primeiro envio do programa precisa ser realizado de maneira convencional, utilizando a conexão física com a placa. Isso é necessário pois o firmware responsável pelo OTA ainda precisa estar previamente gravado no ESP32. Depois desta primeira programação e com o ESP32 conectado na mesma rede do computador, as próximas versões podem ser transferidas pela rede.

Durante uma atualização são exibidas informações pelo monitor serial, incluindo o início da atualização, porcentagem transferida, finalização e possíveis erros. Também foi implementada uma senha para impedir que uma atualização seja iniciada sem a autenticação definida no programa.

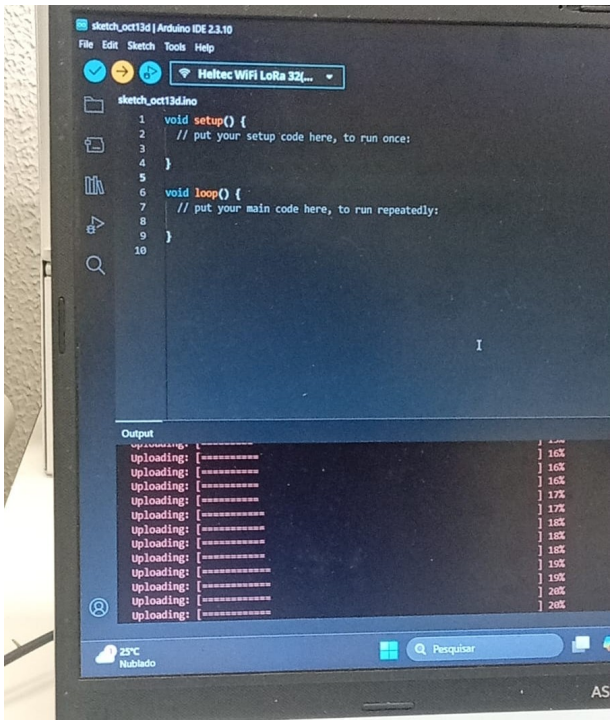


Figura 9: Fotografia do OTA fazendo upload com sucesso.

Uma vantagem observada nesse método é que, após o ESP32 estar instalado e alimentado, pequenas alterações no firmware podem ser feitas sem a necessidade de retirar o dispositivo do local apenas para conectá-lo novamente através do cabo USB.

4.4 Desenvolvimento extra

Além dos requisitos principais do trabalho, foram desenvolvidas algumas funcionalidades adicionais para facilitar os testes e aumentar as possibilidades de utilização do sistema.

O primeiro desenvolvimento foi um simulador completo do ESP32 executado no computador. Ele foi desenvolvido em Python e possui uma API semelhante ao dispositivo físico. O simulador gera valores para temperatura, umidade e pressão, permite alterar o estado de um botão virtual e reproduz também o funcionamento do LED. A cada 5 segundos os valores gerados são enviados para o mesmo *backend* utilizado pelo ESP32 real.

Essa implementação foi bastante utilizada durante o desenvolvimento, pois permitiu construir e testar praticamente todo o restante do sistema antes da utilização do hardware físico.

Também foi desenvolvida uma interface Web utilizando HTML, CSS e JavaScript. A interface permite visualizar as medições dos sensores, estado do botão, situação da conexão e controlar o LED. Para representação gráfica do histórico foi utilizado o Chart.js. Assim como no Node-Red, a comunicação é realizada através da API REST, sem acesso direto ao banco de dados pelo navegador.

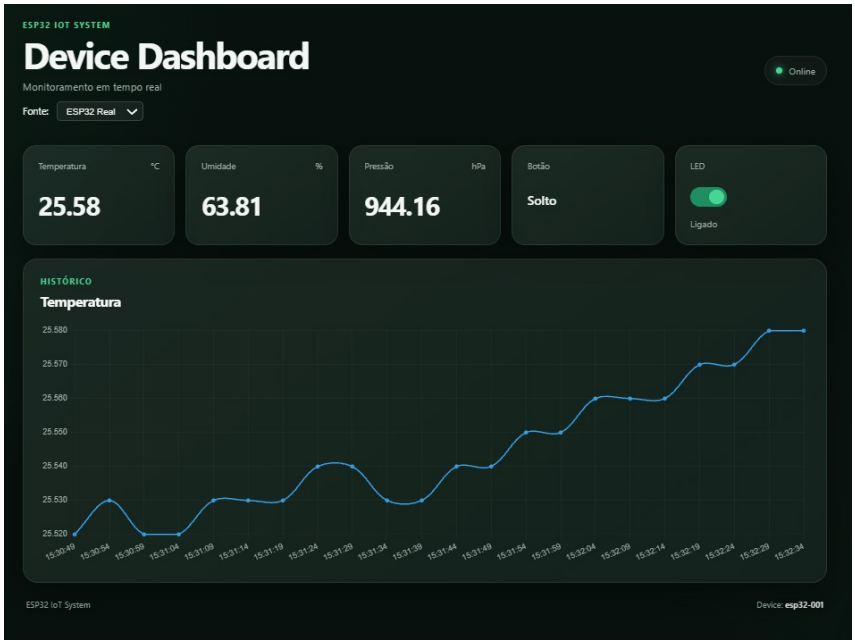


Figura 10: Captura de tela do dashboard web desenvolvido com HTML5

Outro desenvolvimento adicional foi a implementação do banco de dados PostgreSQL. As medições recebidas são armazenadas na tabela `sensor_logs`, enquanto as alterações do LED são registradas em `actuator_status`. Com isso, além de visualizar apenas o valor atual dos sensores, é possível consultar informações coletadas anteriormente.

Foi desenvolvido também o sistema para selecionar qual dispositivo deve alimentar a aplicação. A API mantém uma fonte ativa, que pode ser `simulator` ou `real`. Caso um dispositivo diferente da fonte selecionada tente enviar telemetria, seus dados não são aceitos. Esse controle foi utilizado principalmente para impedir que os valores simulados fossem misturados com as medições obtidas pelo ESP32 real.

5 Conclusão

O desenvolvimento deste trabalho permitiu implementar um sistema completo de comunicação entre um ESP32 e diferentes aplicações através de uma API REST HTTP. A partir do

sensor BME280 foi possível realizar a leitura das variáveis ambientais e enviar essas informações através da rede Wi-Fi, enquanto o LED pôde ser controlado de maneira remota.

A utilização do Node-Red permitiu reunir em uma única interface os dados obtidos, gráficos, estado do botão, controle do LED e informações sobre a conexão do dispositivo. Além disso, a implementação do OTA possibilitou realizar novas programações do ESP32 através da rede, diminuindo a dependência da conexão física com o computador após a primeira gravação.

Durante o desenvolvimento também foram implementadas algumas funcionalidades extras, principalmente o simulador do ESP32, a interface Web e o banco de dados PostgreSQL. O simulador permitiu desenvolver grande parte do sistema mesmo sem o equipamento físico conectado, enquanto o banco de dados possibilitou manter um histórico das medições realizadas.

A criação de uma opção para selecionar entre o simulador e o ESP32 real também facilitou bastante os testes, pois ambos utilizam praticamente a mesma estrutura de comunicação com a API. Dessa forma, foi possível substituir a fonte dos dados sem precisar modificar manualmente todas as requisições utilizadas pelo sistema.

De forma geral, os resultados obtidos atenderam os principais objetivos propostos para o trabalho, sendo possível realizar a aquisição, transmissão, armazenamento e visualização dos dados, além do controle remoto de um atuador. A estrutura desenvolvida também permite que futuramente outros sensores ou atuadores sejam adicionados, utilizando a mesma arquitetura de comunicação já implementada.